



COMP1850 Programming

Week 2, Session 2

Booleans & Selection Statements

Today's Aims

- Understand the features of the `bool` type
- Be able to construct and evaluate **boolean expressions**
- Learn how boolean expressions are used in **selection statements** that choose between different execution paths in a program

The Boolean Type

- `bool` is a specialized version of the `int` type, supporting only two possible values
- These values have special names: `True` and `False`
- Note: **no quotes** (these are not strings)
- Note: **uppercase initial letter** (in other languages such as C, C++, C#, Java and Kotlin, the initial letter is lowercase)

Boolean Expressions

- A **boolean expression** evaluates to True or False
- These expressions often use **comparison operators** to compare two values in some way

Operator	Name
==	equal to
!=	not equal to
>	greater than
>=	greater than or equal to
<	less than
<=	less than or equal to

Task 1

`7 > 4`

`9 < 1`

`8 >= 3`

`"hello" == "Hello"`

`"a" > "e"`

Do these expressions evaluate to True or False?

Predict each result, then try entering the expression in the Python interpreter

Comparison of Floats

```
a = 0.15 + 0.15  
b = 0.1 + 0.2  
a == b          # True or False?
```

Try this out for yourself in the Python interpreter!

(See <https://floating-point-gui.de/errors/comparison/> for more info)

Boolean Operators

Python has three **boolean operators** – and, or, not – that can be used to construct more complex boolean expressions

Expression	Result
x and y	True if both x and y are True, otherwise False
x or y	True if x or y or both are True, otherwise False
not x	True if x is False, False if x is True

Example

We could test whether number is in the range 1 to 10 with

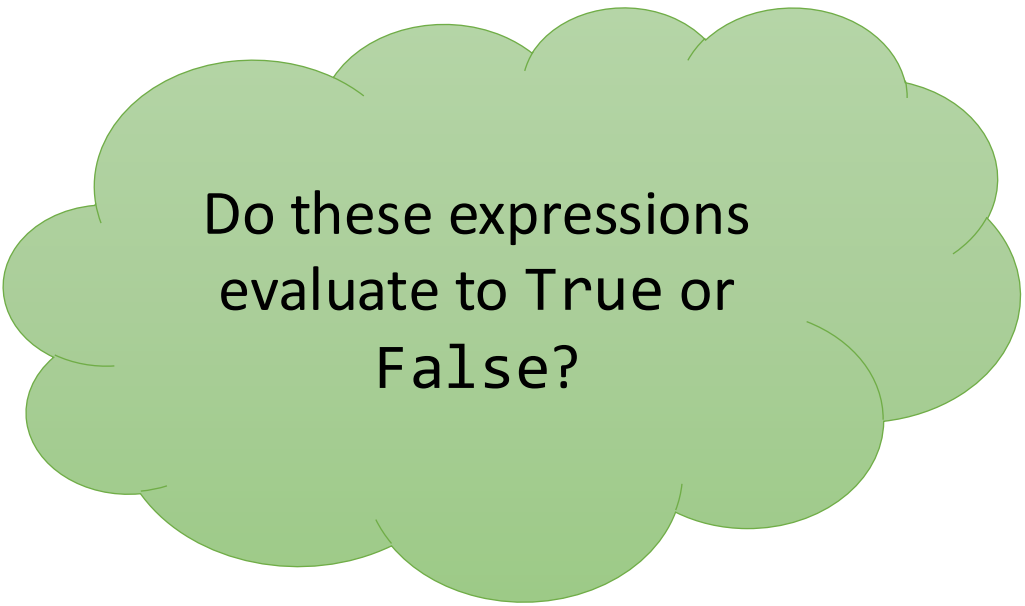
`number >= 1 and number <= 10`

Note: we could avoid use of the and operator here by chaining the comparisons like so:

`1 <= number <= 10`

Task 2

- a) `not (3 > 1)`
- b) `(4 > 2) and (6 > 8)`
- c) `(5 == 5) or (2 > 10)`
- d) `not (10 > 5 or 3 != 3)`
- e) `5 < 10 < 15`
- f) `3 <= 3 < 10 and (10 == 10 or 5 > 8)`



Do these expressions
evaluate to True or
False?

Membership Testing

The `in` operator can be used to construct boolean expressions that test whether an item is part of a string or a collection:

```
"x" in "hello"           # False
"he" in "hello"          # True
3 in [1, 2, 3, 4]        # True
"x" in {"a", "b", "c"}   # False
"c" in {"a": 1, "b": 2, "c": 3} # True (tests keys)
```

Selection Statements

- A boolean expression represents a binary decision
- We can use the outcome of such an expression to choose between different paths of execution in a program
- We call statements that do this **selection statements**
- The simplest example is the **if statement**

Basic If Statement

`if` *boolean-expression*:

Block of code to execute if result is True

IMPORTANT: the block of code must consist of statements with exactly the same level of indentation!

Example

```
password = input("Enter a password: ")  
  
if len(password) < 10:  
    print("Password too short!")  
    print("Minimum password length is 10")
```



Fixed indentation
of 4 spaces



If-Else Statement

`if` *boolean-expression*:

Block of code to execute if result is True

`else`:

Block of code to execute if result is False

Task 3: Simple If Statements

- Go to `week1.2/session2/task3/`
- Edit `task3.py` and finish the program, using the information in the comments as a guide
- Note: you will need to replace `XXX` with an appropriate boolean expression

Scenario

A shop gives a 10% discount when people spend £50 or more.

The till needs to decide whether people qualify for the discount and apply it automatically:

```
if cost >= 50:  
    final_cost = 0.9 * cost
```

But what if the shopkeeper wants to add a *second* discount: if you spend £200 or more you get 20% off. How can we implement this?

Adding More Branches

`if first-boolean-expression:`

Block of code to execute if first result is True

`elif second-boolean-expression:`

Block of code to execute if second result is True

`else:`

Block of code to execute if neither result is True

Task 4: Multiple Branches

- Go to `week1.2/session2/task4/`
- Finish implementing each of the three programs in this directory, using the information in the comments as a guide

Common Errors

- Using $>$ when $<$ was needed (or vice versa)
- Choosing correctly between $>=$ and $>$ (or $<=$ and $<$)
- Using and when or is needed (or vice versa)
- not (A and B) **is not the same as** not A and not B
(see [De Morgan's laws](#))
- Getting indentation wrong in if statements

'Truthy' & 'Falsy' Values

Equivalent to True, in a Boolean context:

- Any non-zero number (e.g., -5, 100, 4.287)
- Non-empty string (e.g., "hello", " ")
- Non-empty collection (e.g., [1, 2, 3], {"age": 42})

Equivalent to False, in a Boolean context:

- 0 and 0.0
- None (Python's version of the null value in other languages)
- Empty string ("")
- Empty collection (e.g., [], {})



Example

```
name = input("Enter your name: ")  
if not name:  
    print("You didn't enter anything!")
```



Task 5: More Practice

- Go to `week1.2/session2/task5/`
- Finish implementing each of the three programs in this directory, using the information in the comments as a guide

Match Statements

- Another way of selecting a branch based on value of a variable
- Similar to switch statements in other languages
- Can be neater and easier to understand than a multi-branch if statement

See [official Python docs](#) for more details...

Example

```
match day:
    case "Monday" | "Wednesday":
        print("Go to gym")
    case "Tuesday" | "Thursday":
        print("Take a walk")
    case "Friday" | "Saturday" | "Sunday":
        print("Do nothing")
    case _:
        # _ matches any value
        print("Error! Invalid day!")
```

Cases can match multiple
options; vertical bar
means 'or' here

Task 6: Something Larger

`week1.2/session2/task6/`

The file `monitor.py` is a larger program that builds on what you have seen already. Read the comments and implement Steps 1 to 4.

If you already know Python fairly well, implement the extension feature described in the comments.

Other things to try:

- Read values from a file
- Colour the output text (e.g., red for danger)



Finding Out More

How To Think Like a Computer Scientist

- [Chapter 7: Selection](#)

Summary

We have

- Looked at Python's `bool` type
- Seen how comparison operators and boolean operators can be used to construct expressions that yield `bool` values
- Used boolean expressions to create **selection statements** that choose between different paths of execution in a program
- Briefly considered an alternative selection structure: `match`